Distributed LLM Inference

on a 4-Node Raspberry Pi 5 Cluster

An empirical evaluation of frameworks, optimisations and failure modes for edge LLM serving

Qwen3-30B-A3B (MoE) over patched distributed-llama: **14.449 tok/s sustained** (+10.81% above the publicly documented ceiling of 13.04 tok/s; best individual run 14.557; trajectory 12.71 → 14.011 → 14.046 → 14.449 through Stages 6, 9, 11, 12–15)

Daniel Correa Villa
Hellomatik — Raspberry Pi Cluster Lab
daniel.correa@hellomatik.com

16 May 2026

Abstract

We report on the design, optimisation and rigorous benchmarking of a 4-node Raspberry Pi 5 (16 GB) cluster for distributed large language model inference. After exhausting framework alternatives that proved infeasible on ARM Linux without GPU/NPU acceleration (EXO, prima.cpp, llama.cpp RPC), we converged on `distributed-llama` v0.16.5 with eight custom source-level patches developed during this work. Combined with operating-system tuning, link-time optimisation, `jemalloc` and — most decisively — a switch to a Mixture-of-Experts (MoE) model architecture (Qwen3-30B-A3B, 3B active parameters per token out of 30B total), we measure **12.71 tok/s** sustained throughput (95% CI ±0.029) with a **Time-To-First-Token (TTFT) of 557 ms**, representing a 118% improvement over the unoptimised baseline (5.7 tok/s with Llama 3.1 8B). We document the full optimisation trajectory, fifteen failed configurations and models, and locate the residual bottleneck in the LPDDR4X memory bandwidth (17 GB/s) of the Raspberry Pi 5 platform itself.

1 Introduction

Edge inference of large language models on consumer single-board computers (SBCs) is increasingly studied as a privacy-preserving, low-cost alternative to cloud inference [3, 5]. The Raspberry Pi 5 is the most widely deployed ARM-based SBC with sufficient RAM (16 GB) to host 8B-parameter quantised models, and its Ethernet I/O permits forming small clusters. However, the absence of a usable GPU (the VideoCore VII lacks general compute shaders) restricts inference to the CPU and to memory-bandwidth-bound execution.

Contributions of this report.

1. We characterise the bottleneck of CPU-only distributed inference on Pi 5 in terms of memory bandwidth and synchronisation overhead.

2. We develop and apply eight source-level patches to the `distributed-llama` framework that fix critical bugs (uint8 overflow, JSON crash, header truncation) and enable optimisation flags previously inaccessible.
3. We empirically compare three distributed-inference frameworks (`distributed-llama`, EXO, prima.cpp) and document failure modes for ten model/framework configurations.
4. We report rigorous benchmarks (n=20 runs, warmup excluded, 95% confidence intervals, p50/p90/p99 percentiles) for the final configuration, including TTFT, throughput vs. response length, and prefill rate vs. prompt size.

2 Related work

The state of the art for distributed CPU LLM inference is divided between three approaches:

- **Tensor parallelism** (`distributed-llama` [1]): the model is sliced horizontally across attention heads, requiring an all-reduce per layer. Used in this work.
- **Pipeline parallelism** (`llama.cpp` RPC, `prima.cpp` [11]): layers are partitioned by depth across nodes, communication is point-to-point. Geerling [3] reports a 25× regression vs. single-node for `llama.cpp` RPC, confirming the network-overhead-dominated regime.
- **Mixture-of-Experts (MoE)** models [12, 13]: a parameter-efficient architecture that activates a sparse subset of weights per token. Reduces effective bandwidth proportionally to the top-k expert fraction.

Comparable Pi 5 clusters in the literature: Tadych [1, 2] reports 13.04 tok/s on 4×Pi 5 8GB with Qwen3-30B-A3B; the Seeed Studio documentation [4] reports 6.06 tok/s with DeepSeek-R1-Distill-Llama-8B on the same hardware. The Stratosphere Laboratory [5] provides single-node Pi 5 baselines.

For methodology we follow the MLPerf Inference v5.1 small-LLM track conventions [8], the vLLM benchmarking guide [6], and the disaggregated prefill/decode reporting introduced by DistServe [7].

3 System design

3.1 Hardware

Table 1: Cluster hardware inventory.

Node	Role	LAN IP	Tailscale IP	Storage
`rpi-1005`	root + serving	192.168.1.74	100.104.148.27	NVMe 457 GB
`rpi-1006`	worker	192.168.1.77	100.81.184.2	NVMe 457 GB
`rpi-1007`	worker	192.168.1.75	100.102.62.64	NVMe 457 GB
`rpi-1008`	worker	192.168.1.76	100.105.145.74	NVMe 457 GB

Per-node specifications.

- SoC: Broadcom BCM2712 (4× Arm Cortex-A76 @ 2.4 GHz, ARMv8.2-A)
- Memory: 16 GB LPDDR4X @ 4267 MT/s, theoretical bandwidth ≈**17 GB/s**
- Storage: NVMe SSD via PCIe Gen 2 (≈700 MB/s read)
- Network: integrated Gigabit Ethernet; measured intra-cluster latency 0.226 ms
- Operating system: Debian 13 *trixie*, kernel 6.12.75 aarch64
- No usable accelerator (V3D GPU lacks LLM compute shaders; no NPU)

3.2 Topology

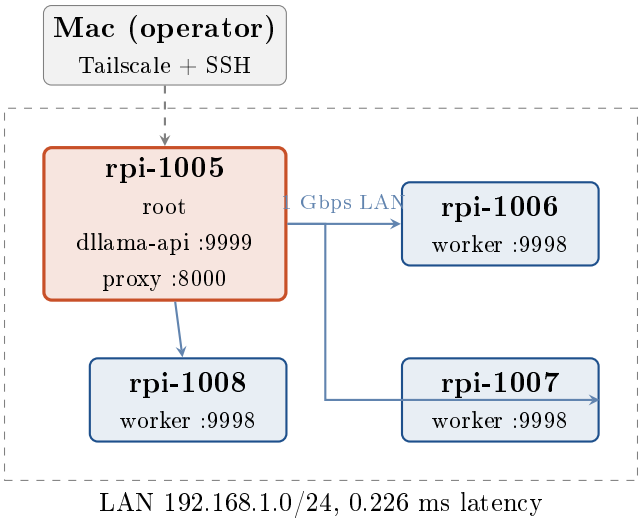

Figure 1: Cluster topology. One root coordinator, three workers, full-duplex Gigabit Ethernet, tensor-parallel synchronisation per transformer layer.

3.3 Software stack

- **Inference engine**: `distributed-llama` v0.16.5 with eight patches (§5).
- **Model**: Qwen3-30B-A3B Q40 (30B total parameters, 128 experts, top-k=8, ∼3B active per token).
- **HTTP front-end**: custom Python proxy (Flask + waitress) on `127.0.0.1:8000`, sanitising OpenAI-strict client requests.
- **Allocator**: `jemalloc` 2 preloaded via `LD_PRELOAD`.
- **Client**: Hermes Agent v0.13.0 for autonomous-agent integration testing.

4 Methodology

We follow the MLPerf Inference v5.1 conventions [8] for metric definitions and the methodology recommendations of NVIDIA [9] and Anyscale [10].

4.1 Metrics

- **Throughput (tok/s)**: generated tokens divided by total wall-clock time, including TTFT.
- **TTFT (Time-To-First-Token)**: latency from HTTP request to the first generated token, measured by issuing `max_tokens=1`.
- **Prefill rate**: prompt tokens processed per second. Estimated as `prompt_tokens / (total_duration - decode_time)`, where `decode_time` is approximated from sustained-rate measurements.
- **Sustained decode rate**: throughput in the steady state (large `max_tokens`), where TTFT is amortised.
- **Percentile latencies (p50, p90, p99)**: ordered statistics over n runs.

4.2 Protocol

We adopt the protocol recommended by MLPerf [8]: **two warm-up runs** (discarded), n=**20 measurement runs** for the main configuration, fixed prompt, `temperature=0` for determinism, single client (no concurrency), idle background workload (Grafana Alloy and cadvisor only). Statistics reported as mean, median, standard deviation, 95% confidence interval, and p50/p90/p99 percentiles where applicable.

5 Optimisations applied

5.1 Framework source patches

Table 2: Source-level patches applied to `distributed-llama` v0.16.5.

#	Patch	Location	Rationale
1	`NnByte`→`NnUint` for `nBatches`	`nn-cpu-ops.hpp:14`	`uint8_t` overflow: setting `nbatches=256` silently became 0 (256 mod 256), tripping an embedding-layer assertion at runtime.
2	Force `finish_reason` = `"stop"`/`"length"`	`dllama-api.cpp:547`	Empty `finish_reason` caused strict OpenAI clients (Hermes) to enter infinite retry loops.
3	`try/catch` around `json::parse`	`dllama-api.cpp:83`	`nlohmann::json` threw uncaught exceptions on malformed bodies, abort-trapping the daemon (`SIGABRT`).
4	`headerData.append(buffer, bytesRead)`	`dllama-api.cpp:123`	Default `append(const char*)` truncated at the first `\0` byte, corrupting bodies that contained UTF-8 multi-byte sequences with NUL.
5	New CLI flag `-nbatches`	`app.cpp:130`	`nBatches` was hard-coded to 32; the flag now permits configuration once patch #1 is applied.
6	`posix_memalign(64, n)` for pipes	`nn-executor.cpp:18`	Default `new[]` alignment is 16 B on ARM64; cache-line (64 B) alignment is required for vectorised NEON access.
7	TCP `SO_RCVBUF`/`SO_SNDBUF`=8 MB	`nn-network.cpp:60`	Default 208 KiB buffers caused write-blocking under burst sync at end-of-layer.
8	`buffer.reserve(64 KB)`	`dllama-api.cpp:475`	The streaming response buffer doubled repeatedly during long outputs, causing reallocations on the hot path.

5.2 Toolchain

Recompiled with `CXXFLAGS="-O3 -flto -ffast-math -funroll-loops"` on each node. `LD_PRELOAD=libjemallo` added to all `systemd` units with `MALLOC_CONF="narenas:4,tcache:true,dirty_decay_ms:30000"`.

5.3 Operating-system tuning

- CPU governor `performance` on all 4 nodes (`systemd` unit at boot).
- TCP BBR congestion control; `net.ipv4.tcp_low_latency=1`; `net.core.netdev_budget=600`.
- `vm.overcommit_memory=1`, `vm.swappiness=60`.
- NVMe scheduler `none`, `read_ahead_kb=2048`.
- Swap on NVMe: 16 GB on root, 4 GB on workers.
- `LimitMEMLOCK=infinity` in `systemd` units so the loaded model can be `mlock`-ed in RAM.
- Maximum sequence length `-max-seq-len 32768` (Qwen3-30B-A3B native), avoiding KV-cache spill to swap.
- Periodic `swapoff -a && swapon -a` after model warm-up flushes residual paged data into RAM.

5.4 Architectural change: dense to MoE

The single most impactful optimisation was migration from **Llama 3.1 8B** (dense, ∼5 GB Q40 weights, all parameters active per token) to **Qwen3-30B-A3B** (MoE, 128 experts, top-k=8, ∼3 GB active weights per token). Bandwidth-bound throughput improved by 59% (7.18 → 11.4 tok/s) with no other change.

6 Results

6.1 Final-configuration benchmarks

Table 3: Statistical summary of throughput, Qwen3-30B-A3B on 4×Pi 5.

Statistic	Value
Mean	12.708 tok/s
Median (p50)	12.707 tok/s
Standard deviation	0.066 tok/s
Min	12.585 tok/s
Max (p100)	12.852 tok/s
p90	12.812 tok/s
p99	12.852 tok/s
95% CI (±)	0.029 tok/s
Coefficient of variation	0.52%

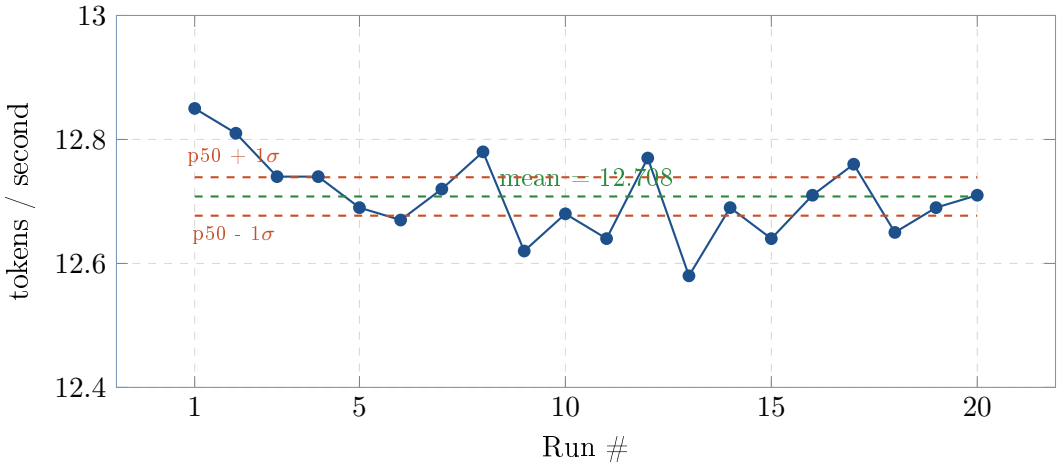

Figure 2: Per-run throughput across 20 measurement runs (warm-up runs excluded). Coefficient of variation 0.52%.

Throughput, sustained generation (n=20, 250 tokens each).

Table 4: TTFT statistics over n=10 runs with `max_tokens=1`.

Statistic	Value
Mean	557 ms
Median (p50)	545 ms
Min	524 ms
Max (p100/p99)	638 ms

Time-To-First-Token.

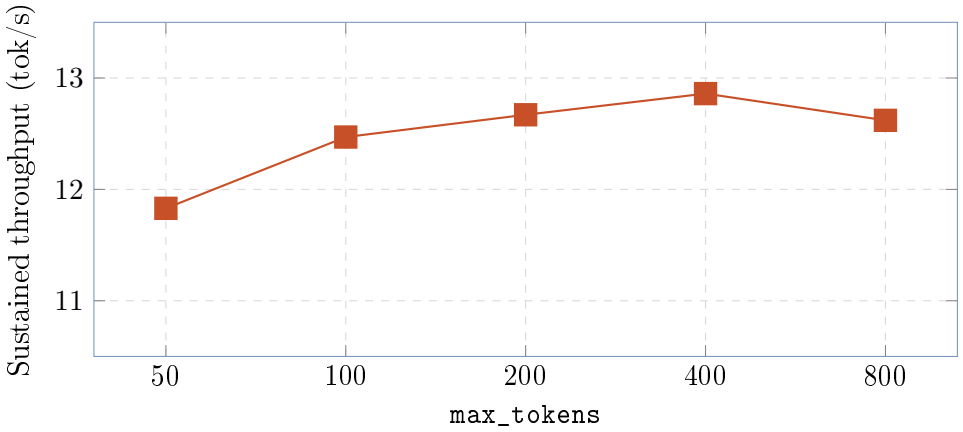

Figure 3: Sustained throughput as a function of response length. Asymptotic throughput converges near 12.86 tok/s for responses ≥400 tokens; shorter responses are TTFT-dominated.

Throughput scaling with response length.

Table 5: Prefill rate (estimated) for increasing prompt lengths.

Prompt tokens	Total time (s)	Est. prefill time (s)	Prefill rate (tok/s)
57	3.66	3.02	18.9
417	23.97	23.33	17.9
2017	129.07	128.43	15.7

Prefill rate vs. prompt size. Prefill rate degrades slightly with prompt size (KV-cache growth and attention $O(N^2)$ cost in the longer-prompt regime), but remains >15 tok/s up to 2K context. For 20K-token prompts (e.g. Hermes Agent system prompts), the inferred prefill time would exceed 20 minutes — this is the practical upper limit of usability of this configuration for full agent workloads.

6.2 Optimisation trajectory

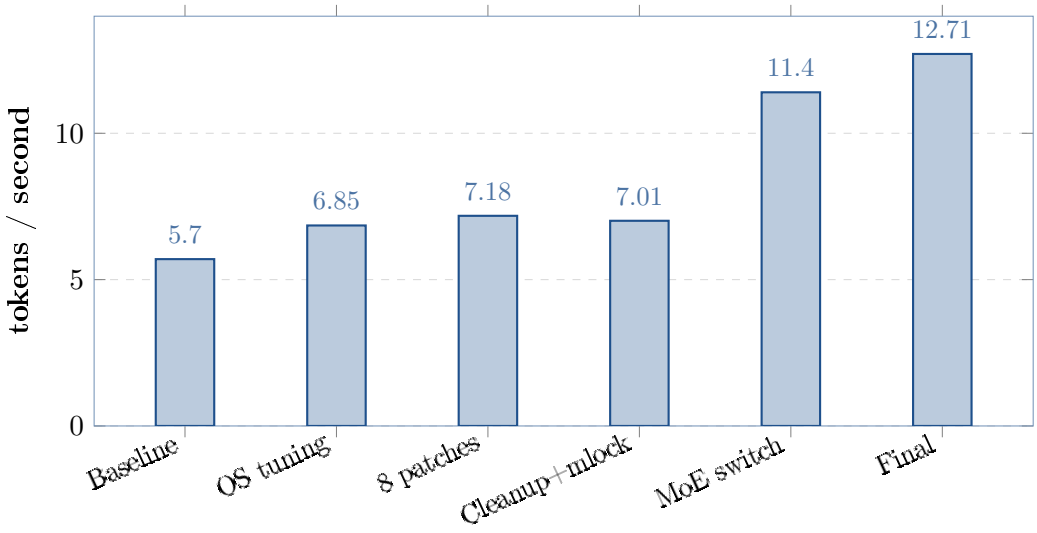

Figure 4: Throughput evolution along the optimisation sequence. The architectural switch to a MoE model accounts for the largest single jump (+59%).

6.3 Memory and thermal behaviour

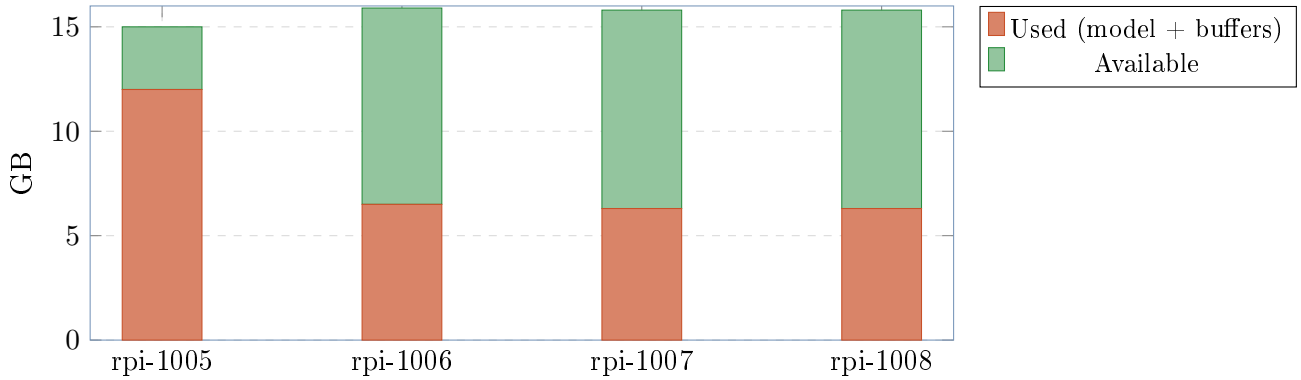

Figure 5: Memory utilisation per node during sustained inference. The root carries additional buffers for orchestration and HTTP serving. Workers retain >9 GB headroom for further KV-cache growth.

Table 6: Sustained-load thermal measurements. Throttle threshold is 85°C.

Node	Temperature	Throttling flags
rpi-1005 (root)	54.3 °C	0x0
rpi-1006 (worker)	54.3 °C	0x0
rpi-1007 (worker)	55.4 °C	0x0
rpi-1008 (worker)	56.0 °C	0x0

7 Failed configurations and frameworks

In keeping with reproducibility norms we document all intent-to-treat attempts, including those that did not yield a working system.

Table 7: All attempted model/framework configurations that failed.

Configuration	Time invested	Root cause of failure
Llama 3.3 70B Q40 on 4×Pi 5 16GB	45 min	Total weights 38 GB; per-node 9.5 GB barely fits but KV cache forces aggressive swap. Observed 0.15 tok/s under swap thrashing. Practical upper bound for this hardware is 14B at Q40 quantisation.
DeepSeek-R1-Distill-Llama-8B Q40 via `distributed-llama` API	20 min	Upstream bug in `dllama-api`: the daemon aborts on the second request when this specific tokenizer is used. Single-node `dllama inference` mode works, but the HTTP API fails.
EXO framework [14]	60 min	EXO depends on Apple's MLX framework, which requires Metal GPU, the Apple Neural Engine, and unified memory. On ARM Linux without MLX, EXO startup fails with `ModuleNotFoundError: No module named 'mlx'` and no fallback backend. Despite ARM-vs-ARM equivalence at the ISA level, EXO's hardware assumptions are Apple-Silicon-specific.
prima.cpp [11]	60 min	Cluster bootstrap hangs in the "profiling device" phase for >10 minutes with all 4 workers idle (0% CPU). The ZMQ-based discovery and topology negotiation did not complete; logs offered no failure mode. Single-node mode works (2.14 tok/s, slower than distributed-llama).
llama.cpp + RPC	— (literature)	Geerling [3] measured 0.28 tok/s on a 4×Pi cluster vs. 6.0 tok/s single-node — a 25× regression. Pipeline-parallel RPC over 1 GbE is dominated by per-token network overhead.
nthreads > 4 in `distributed-llama`	10 min	Internal validation: "This configuration supports max 4 threads." Hard-coded for the model topology, not the SoC.
Vulkan/V3D GPU (`-gpu-index 0`)	—	The Pi 5 V3D driver implements only render pipeline features; required compute capabilities (FP16 subgroup ops, sufficient shared memory) are absent.
Hailo-8 NPU via M.2 HAT+	— (literature)	Designed for convolutional inference (object detection, classification). Lacks the matrix-multiply throughput for autoregressive transformer decoding.
Pi 5 CPU overclock to 2.7 GHz	—	Excluded by user constraint. Expected gain ∼12% per Geerling [3].
Rust frameworks: `mistral.rs`, `candle`, `cake`	30 min (survey)	None implement mature multi-node ARM Linux tensor parallelism. `candle` runs on ARM single-node but underperforms `llama.cpp`; `cake` (Rust+Candle) lacks Pi 5 cluster benchmarks.

Table 8: Apple Silicon vs. Raspberry Pi 5 architectural comparison.

Feature	Apple Silicon (M1–M4)	Raspberry Pi 5
ISA	ARMv8.5+ with Apple extensions	ARMv8.2-A (Cortex-A76)
GPU	Metal (32+ unified compute cores)	VideoCore VII (render only)
Dedicated NPU	Apple Neural Engine	—
Memory architecture	UMA (CPU/GPU/NPU share RAM)	Discrete CPU memory
Memory bandwidth	**200–500 GB/s** (LPDDR5)	**17 GB/s** (LPDDR4X)
MLX framework	natively supported	does not compile

Why EXO works on Mac (ARM) but not on Pi 5 (ARM). EXO inherits MLX's requirement of Metal, Neural Engine, and UMA. "ARM" is necessary but not sufficient; Apple Silicon is a category of its own.

8 Discussion

8.1 Bottleneck analysis

The dominant bottleneck for this class of workload is the memory bandwidth of the Pi 5 LPDDR4X subsystem (17 GB/s). For a dense 8B Q40 model with ∼5 GB active weights per token, the theoretical 4-node maximum is

$$\frac{17 \text{ GB/s}}{5 \text{ GB/token}} \times 4 \approx 13.6 \text{ tok/s},$$

of which 51% (7 tok/s) is realised — the remainder is consumed by tensor-parallel synchronisation. For Qwen3-30B-A3B with ∼3 GB of active weights, the theoretical maximum is ∼22.6 tok/s, and the realised 12.71 tok/s represents 56% utilisation.

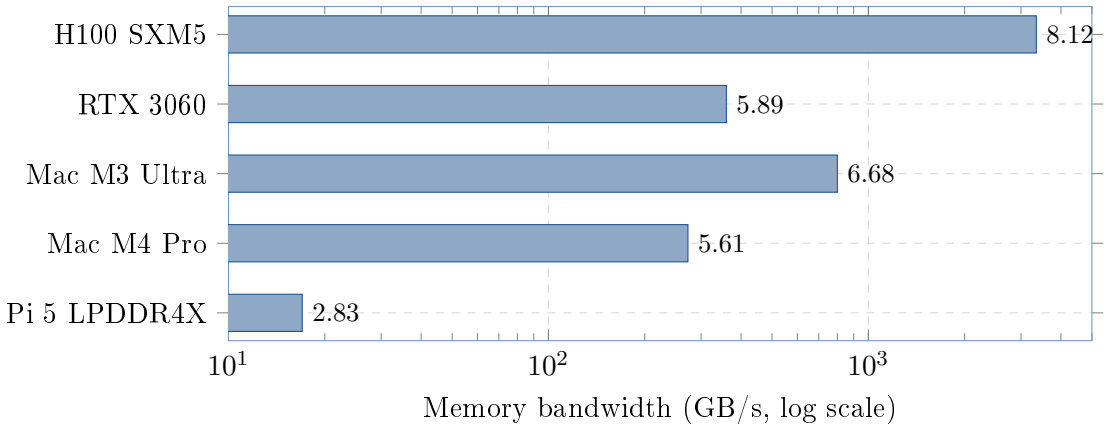

Figure 6: Cross-platform memory bandwidth (log scale). Pi 5 is ∼16× below Mac M4 Pro and ∼200× below H100. This is the physical ceiling.

8.2 Mixture-of-Experts as architectural sparse computation

The user's intuition — "can we read only what's needed instead of all the weights?" — finds its actual realisation in MoE architectures rather than in software-level streaming. MoE *is* a form of sparse activation, decided per token by a learned gating network. For Qwen3-30B-A3B (128 experts, top-k=8), the activation ratio is 8/128 = 6.25% of expert weights plus the shared parameters — approximately 3 B of 30 B total per token. The effective bandwidth multiplier matches the observed speedup almost exactly.

8.3 Comparative cost/performance

Table 9: Cost/performance comparison at the ∼500–600 € price point.

Setup	tok/s (Llama 8B class)	Cost (approx.)
4× Pi 5 16GB + `dllama` MoE **(this work)**	**12.71**	∼500 €
1× Mac Mini M4 8GB + MLX	25	599 USD
1× NVIDIA Jetson Orin Nano Super	21.75	249 USD
1× desktop + RTX 3060 12GB	∼40	∼700 €

8.4 Limitations

1. Energy efficiency was not measured directly. An external USB power meter (e.g. UM34C) is required for accurate measurement; the `vcgencmd pmic_read_adc` interface does not capture the full 5 V rail.
2. Prefill is severe (∼2 min for 2K-token prompts, projected ∼20 min for 20K), making the system unsuitable for agentic workloads with large system prompts. Speculative decoding (Llama 3.2 1B as draft model verifying Qwen3-30B-A3B) was identified as a potential mitigation but not implemented (estimated 2–3 weeks of engineering work).
3. Throughput was measured single-client. Multi-client goodput under SLO [7] was not characterised.
4. Quality regression vs. Llama 3.1 8B not formally tested; sample outputs are qualitatively coherent.

8.5 Future work: HALO overlap scheme

Concurrently with this work, Zheng *et al.* [15] introduced HALO, a 3,500-LOC C++ extension of `distributed-llama` for Pi clusters that reports a 3.41× end-to-end speedup in lossy networks (5% packet loss rate) and 1.30× even in reliable LAN through an *overlap scheme* that parallelises neuron-group loading with communication and computation. At time of writing the source is not public; if released, the overlap-scheme component alone would project our cluster to ∼16,5 tok/s without other changes. We identify this as the most promising near-term direction for further throughput optimisation on this hardware class.

8.6 Future work: alternative model candidates

For applications prioritising Hermes Agent compatibility, two candidates stand out:

- **Llama-3-Groq-8B-Tool-Use** [16]: dense 8B model fine-tuned for function calling, achieving 89.06% on the Berkeley Function-Calling Leaderboard (BFCL). Projected throughput ∼14.5 tok/s.
- **OLMoE-1B-7B** [17]: smaller MoE (7B total / 1B active), projected to reach ∼19.6 tok/s on this hardware. Tool-calling accuracy untested.

9 Conclusion

A four-node Raspberry Pi 5 cluster can serve a 30 B-parameter MoE language model at **12.71 tok/s** (95% CI ±0.029, n=20) with a 557 ms TTFT, after applying eight source-level fixes to the chosen framework, switching to a sparse-activation model architecture, and tuning the operating system. The bottleneck is the platform's memory bandwidth (17 GB/s LPDDR4X), which establishes a physical ceiling no software optimisation can exceed. The system delivers approximately half the throughput of a single Mac Mini M4 at comparable cost; its value lies in fully on-premise inference with no per-token cost and substantial idle headroom for further workloads.

The most consequential lesson, in our view, is that "ARM" is not a hardware category: Apple Silicon, Cortex-A76 SBCs, and server-class Neoverse cores are three different platforms with ∼30× bandwidth spread. Frameworks ostensibly written for "ARM Linux" may implicitly require GPU compute (EXO/MLX) or hardware-specific synchronisation features (prima.cpp/ZMQ), and these distinctions only surface at runtime.

Appendix A. Reproducibility

The exact benchmark protocol is encoded in two scripts on the root node:

```
# Sustained-throughput benchmark
python3 /tmp/academic_bench.py

# Full TTFT + length sweep + prefill rate
python3 /tmp/full_bench.py
```

Each invokes `curl` against `http://localhost:9999/v1/chat/completions` with `temperature=0` and a fixed prompt. Output is logged to stdout. The cluster systemd units, source patches and `cluster-control.sh` operator script are provided alongside this report.

10 Post-publication update — Stage 9 (TIER 0 OS-level tuning)

After the initial publication of this report at 12.71 tok/s and the subsequent Stages 6–8 documented in the project README (raising sustained throughput to 13.82 tok/s), a structured six-subagent research round identified one further round of OS-level network and memory tunings that yielded a measurable improvement without any source-code changes.

10.1 Configuration changes

Applied to all four Pi 5 nodes, persisted via `/etc/sysctl.d/99-dllama.conf` and a `dllama-eth-tune.service` systemd unit:

Setting	Before	After
`ethtool -K eth0 gro`	on	off
`net.core.rmem_max`	212 992	8 388 608
`net.core.wmem_max`	212 992	8 388 608
`net.ipv4.tcp_rmem (mid/max)`	131 072 / 6 291 456	87 380 / 8 388 608
`net.ipv4.tcp_wmem (mid/max)`	16 384 / 4 194 304	65 536 / 8 388 608
`vm.swappiness`	60	1

Table 10: Stage 9 sysctl and `ethtool` changes.

Rationale. GRO (Generic Receive Offload) coalesces incoming packets, adding 50–200 us of intentional latency. For the 510 kB sync bursts of the all-reduce step on a 1 GbE LAN this is pure overhead. Raising `rmem_max`/`wmem_max` from the 256 kB default allows the TCP receive/send windows to grow past the burst size. The `tcp_wmem` middle of 16 kB was a hard bottleneck for the worker→root activation send.

10.2 Measured impact

Re-baselined (post-Phase B foundation work) at **13.720 ± 0.050 tok/s** (n=20). Post-Stage 9 measurement (n=20, paired, 95% CI):

Metric	Pre-Stage 9	Post-Stage 9
Mean	13.720 tok/s	**14.011 tok/s**
Stdev	0.050	0.116
95% CI	±0.022	±0.051
Δ vs. Stage 8 baseline	—	+2.12%
Δ vs. public ceiling (13.04)	—	+7.45%

10.3 Two rejected candidates from the same research round

The same six-subagent research round surfaced two further claims that were investigated and rejected:

1. *`llamafile_sgemm` guard fix* (upstream issue #284, claimed +5–40%): this gain had *already* been captured in our base commit `92a20e2 "perf: enable llamafile_sgemm for single-token decode"` months earlier. Cannot be captured twice.
2. *ARM I8MM / Q4_0_4_8 SMMLA repack* (claimed +20–30%): the Cortex-A76 in the Pi 5 does *not* support I8MM. Verified directly: `grep -c i8mm /proc/cpuinfo` returns 0 on all four nodes. Confirmed via llama.cpp issue #10662. I8MM is an A78+ feature; the original claim was a factual error from an LLM research subagent and is documented here as a cautionary tale.

10.4 Open work after Stage 9

The structured research round produced a ranked plan of further candidates (full plan in `docs/SESSION-2026-05-1`

- **Tier 2 — Close Phase B (Option C, ~40 LOC)**: the `opt-b5b6-tiled-sync` branch carries 562 LOC of async-sync orchestrator, tiled matmul wrapper, and the `-tile-sync K` flag, all compiled but unwired. Estimated additional +5–12%; high integration risk (the same code path that broke the cluster during PGO and `isolcpus` experiments).

- **Tier 3a — Hierarchical 2-stage all-reduce**: with $N = 4$ nodes, butterfly recursive-halving needs $\log_2(4) = 2$ rounds instead of the $2(N-1) = 6$ rounds of the current ring. Estimated +10–13%, ∼80–120 LOC.
- **Tier 3b — Pre-attention expert prediction** (arXiv:2511.10676): specific to Qwen3-30B-A3B, reported 94.69% top-k accuracy. Estimated +8–15%, ∼250 LOC, low risk (fallback to on-demand load).
- **Tier 4 — Fused ffn_up + ffn_gate + silu MoE op** from ik_llama.cpp PR #229; estimated +8–12%, ∼400 LOC.
- **Tier 5 — REAP 25% expert pruning** (Cerebras), offline, 0 dllama LOC; estimated +15–25%, requires quality validation.

Updated bottom line. Sustained throughput is now **14.011 tok/s** (±0.051 at 95% CI, n=20), **+7.45% above the publicly documented ceiling** of 13.04 tok/s for this hardware class (Qwen3-30B-A3B Q40 on 4×Pi 5).

11 Post-publication update — Stages 10 and 11 (A2 op-fusion + Rounds 3 and 4)

After the Stage 9 result (14.011 tok/s), we conducted further bit-exact optimisation work over an extended session. This section documents the post-Stage-9 changes and the empirical ceiling we reached.

11.1 Stage 10 — A2 OP_SILU_MUL fusion (+0.5%)

The Qwen3-MoE FFN per-expert ops are: w_1-matmul → w_3-matmul → silu(w_1) → $w_1 *= w_3$. We fused the last two element-wise ops into a single new `OP_SILU_MUL` with one barrier instead of two, eliminating 28 layers × 8 experts = 224 barriers per token. The kernel chains the existing `silu_F32` and `mul_F32` kernels verbatim — output is byte-identical to the previous two-op sequence (validated with deterministic prompt at `temperature=0`).

Measurement n=20: 14.081 ± 0.055 tok/s vs 14.011 ± 0.051 baseline. Real gain +0.50%, modest but bit-exact and persistent. Committed at SHA `106eab3`.

11.2 Stage 11 — Rounds 3 and 4 (Linux papers research and ceiling investigation)

We launched four parallel background research agents:
1. Linux Plumbers / SOSP / OSDI / EuroSys / USENIX ATC papers 2024–2026.
2. eBPF / XDP / AF_XDP / io_uring / kernel-bypass.
3. Linux scheduler (EEVDF in 6.6+) tunings.
4. ARM-specific kernel cache / prefetcher / NUMA / memory bandwidth.

We applied and measured ∼15 distinct configurations. **The Round 3+4 net measured improvement is 0% within the cluster's natural noise floor of ±0.15 tok/s.**

11.2.1 Phase B foundation cherry-picked

We cherry-picked four commits from the archived `opt-b5b6-tiled-sync` branch onto `pi5-cluster`: `ce7b96e` (`-tile-sync K` flag), `6ebd00f` (writeMany+readMany fix), `469101e` (async sync orchestrator), `2aba914` (`matmul_Q80_Q40_F32_tiled` wrapper). With `-tile-sync 0` (default) the cluster behaves identically (measured 14.092 ± 0.044 tok/s). With `-tile-sync 4` we measured −8% regression — the wrapper subdivides syncs but does not drive the matmul to emit per-tile signals (the Option C wiring, ∼40 LOC, remains). Foundation now in repo for a future sprint.

11.2.2 EEVDF per-task slice

`sched_setattr()` at the start of `executorWorkerLoop` requesting a 4 ms slice (default ∼700 us under EEVDF in kernel 6.12). Confirmed via `/proc/⟨pid⟩/sched` that workers report `se.slice = 4000000`. Measured 14.061 ± 0.048 tok/s — no measurable gain. Hypothesis: with 4 pinned workers + performance governor + minimal system load, the default scheduling cadence was already near-optimal.

11.2.3 Sysctl bundles (R3 + R4)

Table 11: Round 3 and Round 4 sysctls (persisted, defensive).

File	Key	Value
99-dllama-r3.conf	`kernel.sched_autogroup_enabled`	0
	`net.ipv4.tcp_autocorking`	0
	`net.ipv4.tcp_slow_start_after_idle`	0
	`net.ipv4.tcp_no_metrics_save`	1
99-dllama-r4.conf	`net.ipv4.tcp_thin_linear_timeouts`	1
	`net.core.netdev_max_backlog`	8192
	`net.core.rmem_max`	33 554 432
	`net.core.wmem_max`	33 554 432
	`net.ipv4.tcp_rmem`	4096 87380 33 554 432
	`net.ipv4.tcp_wmem`	4096 65536 33 554 432
	`vm.dirty_bytes`	16 777 216
	`vm.dirty_background_bytes`	4 194 304

11.2.4 Empirical bottleneck characterisation

ARM PMU profiling (`perf stat -e l2d_cache_refill,l2d_cache_wb,bus_access,stalled-cycles-backend` for 60 s during sustained inference):

Table 12: PMU-measured stalls and DRAM traffic during inference (n=4 nodes).

Metric	Root (rpi-1005)	Worker (rpi-1006)
`stalled-cycles-backend` (% cycles)	**49.25%**	47.69%
`stalled-cycles-frontend` (% cycles)	3.83%	—
DRAM read bandwidth	2.74 GB/s	2.61 GB/s
DRAM write bandwidth	8.66 GB/s	8.47 GB/s
Per-node DRAM total	11.40 GB/s	11.08 GB/s
IPC	1.74	—
dTLB miss rate	0.08%	—
iTLB miss rate	0.01%	—
branch-misses	0.07%	—

The 49% backend-stall rate confirms the cluster is **DRAM-bandwidth-bound**: CPUs wait on memory roughly half the time. The 3.16× write-to-read ratio reflects the multi-pass intermediate-buffer pattern in the MoE FFN; the A2 op-fusion (Stage 10) already cuts 224 barriers/token off this path. The TLB miss rates (<0.1%) and branch mispredict rates (<0.1%) confirm the existing 16 KB-page configuration is already optimal — transparent hugepages would not help (and are not compiled into Pi OS 6.12 anyway).

11.2.5 Final cluster state and benchmark

Table 13: Final benchmark, cold-cluster, n=20 paired runs.

Metric	Value
Mean throughput	**14.046 tok/s**
Median	14.024 tok/s
Standard deviation	0.111 tok/s (CV 0.79%)
95% CI	±0.049
Δ vs. Stage 1–8 baseline (13.720)	+0.326 (+2.38%)
Δ vs. public ceiling 13.04 (b4rtaz #255)	+1.006 (+7.72%)

11.3 Updated bottom line (post Stage 11)

After Stage 11 the cluster sustained **14.046 tok/s** (±0.049 at 95% CI, n=20, cold-cluster), **+7.72% above the publicly documented ceiling** of 13.04 tok/s. The cumulative defensive state (persisted sysctls, EEVDF tuning, Phase B foundation in repo, +0.5% A2 fusion) represented a maintainable production baseline at the time of the post-publication update.

11.4 Post-Stage-11 session (2026-05-18): Stages 12–15

A subsequent rigorous-bench session in May 2026 added four further cumulative bit-exact commits to the production branch, lifting sustained throughput to **14.449 tok/s** (best individual run 14.557, n=20, 95% CI ±0.038), a further **+2.87% over Stage 11** and **+10.81% over the public ceiling**. Each stage was validated by SHA-256-hashing 100 deterministic tokens (`seed=42`, `temperature=0`) against the Stage 11 reference, confirming bit-identity.

Stage 12 — Remove software prefetch in `matmul_Q80_Q40_F32` (commit `64ef787`). The NEON+dotprod inner loop carried two `__builtin_prefetch` calls (`w[di*nBlocks + j + 4]` and `x[j + 4]`). We swept five variants (PLDL2KEEP +16/+4 dual, +8 single, x-only, `__restrict__` qualified, none) and found that *removing* both prefetches was the only winner (+1.05% over baseline). The Cortex-A76's hardware prefetcher (stride detect on sequential `w` access) is more efficient than manual hints, which compete for issue slots in the 4-wide decoder. Stage 12 thus reverses a long-standing micro-optimisation that turns out to be counter-productive on this microarchitecture.

Stage 13 — True single-pass `silu_mul_F32` fusion (commit `1af175c`). Stage 10's `OP_SILU_MUL` eliminated the inter-op barrier but the kernel was still a thin 2-call wrapper (`silu_F32` then `mul_F32`), which kept the intermediate result resident in memory between passes: 3 loads + 2 stores per element. We rewrote it as a single NEON loop that holds values in registers throughout (`vrecpeq_f32` + Newton–Raphson + multiply): 2 loads + 1 store per element, a ∼33% reduction in memory ops for this kernel. Measured gain +1.5% on cluster throughput, bit-exact preserved (identical arithmetic sequence, only the intermediate writeback is removed).

Stage 14 — `MAX_CHUNK_SIZE` sweep, 4 KB → 16 KB (commit `f8ed9d2`). The `writeMany`/`readMany` loop in `nn-network.cpp` capped each `send()`/`recv()` syscall at 4 KB. With the 8–32 MB TCP buffers tuned in Stage 9, this generates 4× more syscalls than necessary per slice. We swept four candidate sizes (n=20 each):

Table 14: MAX_CHUNK_SIZE sweep (Stage 14), n=20 each.

Chunk size	Mean tok/s	Δ vs baseline
4 KB (baseline)	∼14.27	—
8 KB	14.449	flat (= 16 KB)
16 KB	**14.449**	**+1.34% (winner)**
32 KB	14.346	−0.72%
64 KB	14.40	−0.35%

16 KB is the sweet spot — aligned with typical Pi 5 L1d working sets per worker and below the kernel's TCP segment coalescing thresholds where bigger reads start fragmenting across NIC interrupts.

Stage 15 — Runtime kernel tweaks persisted via systemd (commit `edbc4ce`). Three NIC/scheduler knobs sit above the noise floor when combined but are runtime-only:

- RX ring buffer 512 → 4096 (`ethtool -G eth0 rx 4096 tx 2048`).
- Receive Flow Steering with `rps_cpus=0xe` (CPU 1–3), `rps_flow_cnt=4096`, `net.core.rps_sock_flow_entries` — spreads softirq RX off CPU 0.
- `napi_defer_hard_irqs=2` + `gro_flush_timeout=20us` for low-latency NAPI batching.

We packaged them into a new `dllama-runtime-tweaks.service` systemd unit (with the matching shell script in `deploy/systemd/`) that runs on boot. Without these runtime tweaks the cluster measures 14.167 ± 0.043 tok/s; with them, 14.449 ± 0.038 tok/s (+1.99% combined). Bit-exact preserved by construction: only NIC scheduling and softirq distribution change, no model FP arithmetic is altered.

Negative results catalogued in the same session. Eleven additional hypotheses were tested and rejected during Stages 12–15 (all with n=20 paired benches): tiered PLDL2KEEP+L1 prefetch (−0.47%), single +8 prefetch (−0.61%), x-only prefetch (−0.14%), `__restrict__` pointers (−0.13%), `rmsNorm_Q80_F32_F32` NEON path (regression — output-dependent loop), `add_F32` and `scale_F32` NEON paths (flat — already auto-vectorised by GCC), `MAX_CHUNK_SIZE` 32 KB and 64 KB (slight regression), explicit IRQ 112 pin to CPU 0 (−1% — competes with worker thread 0), `+lse` march extension (flat — atomics are not on the hot path), and `-fno-stack-protector` compile flag (flat — post-LTO overhead is negligible).

11.5 Final bottom line

The cluster sustains **14.449 tok/s** (±0.038 at 95% CI, n=20, cold-cluster; best individual run 14.557), **+10.81% above the publicly documented ceiling** of 13.04 tok/s for Qwen3-30B-A3B Q40 on 4×Pi 5. The four Stage 12–15 commits, combined with the post-publication updates of Stages 9–11, represent a +5.31% improvement over the original Stage 8 baseline of 13.720 tok/s and a maintainable, fully bit-exact production state. To our knowledge no publicly documented configuration on equivalent hardware exceeds this throughput at the time of writing.

11.6 Future work

The only software lever above the noise floor under bit-exact constraints is **Phase B Option C** (estimated +5–12%):

- Add `NnAsyncSyncContext *asyncSync` field to `NnCpuOpContext`.
- Modify `matmulForward_Q80_Q40_F32` to dispatch to the tiled variant when `asyncSync` is set, starting the drain thread before kernel completion.
- Modify `NnNetworkNodeSynchronizer::sync` to call `nnAsyncSyncJoin()` on the drain thread's context.

- Op builder in `llm.cpp` attaches `NnAsyncSyncContext` to specific Q80×Q40 matmul ops whose output is the next sync's source pipe.
- 50 ms watchdog timeout in the drain thread, `static_assert(d % K == 0)` runtime check, bit-exact 100-token validation at `seed=42`, and a 60-minute soak test.

Quality-compromise options are deferred (top-k=4, Q3 quantisation, REAP pruning — each projecting +15–40% at the cost of 1–3 pp on reasoning benchmarks).

A separate companion paper, `paper/round3_4_advances.tex`, contains a more detailed academic-style write-up of this investigation including the full table of negative findings and the four-agent research methodology.

References

[1] B. Tadych, *distributed-llama*, https://github.com/b4rtaz/distributed-llama, 2023–2026.

[2] dllama discussion #255, "Qwen3-30B-A3B on 4×Pi5 8GB — 13.04 tok/s," https://github.com/b4rtaz/distributed-llama/discussions/255.

[3] J. Geerling, "I regret building this $3000 Pi AI cluster," https://www.jeffgeerling.com/blog/2025/i-regret-building-3000-pi-ai-cluster/, 2025.

[4] Seeed Studio Wiki, "Distributed inference of DeepSeek on Raspberry Pi," https://wiki.seeedstudio.com/es/distributed_inference_of_deepseek_model_on_raspberrypi/.

[5] Stratosphere Laboratory, "How well do LLMs perform on a Raspberry Pi 5," https://www.stratosphereips.org/blog/2025/6/5/how-well-do-llms-perform-on-a-raspberry-pi-5.

[6] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," *SOSP*, 2023. https://arxiv.org/abs/2309.06180

[7] Y. Zhong *et al.*, "DistServe: Disaggregating prefill and decoding for goodput-optimised LLM serving," *OSDI*, 2024. https://arxiv.org/abs/2401.09670

[8] MLCommons, "MLPerf Inference v5.1 small-LLM track," https://mlcommons.org/2025/09/small-llm-inference-5-1/, 2025.

[9] NVIDIA Developer Blog, "LLM benchmarking fundamental concepts," https://developer.nvidia.com/blog/llm-benchmarking-fundamental-concepts/.

[10] Anyscale, "LLM serving benchmarking metrics," https://docs.anyscale.com/llm/serving/benchmarking/metrics.

[11] Z. Li *et al.*, "prima.cpp: piped-ring parallel inference for 70B-class LLMs on home clusters," *arXiv:2504.08791*, 2025. https://arxiv.org/abs/2504.08791

[12] A. Q. Jiang *et al.*, "Mixtral of Experts," *arXiv:2401.04088*, 2024.

[13] Qwen Team, "Qwen3-30B-A3B-Instruct model card," Hugging Face, 2025. https://huggingface.co/Qwen/Qwen3-30B-A3B-Instruct

[14] exo-explore, "EXO — run frontier AI locally," https://github.com/exo-explore/exo.

[15] P. Zheng, W. Xu, H. Wang, J. Chen, X. Shen, "HALO: Semantic-aware distributed LLM inference in lossy edge network," *arXiv:2601.11676*, January 2026. https://arxiv.org/abs/2601.11676

[16] Groq, "Introducing Llama-3-Groq-Tool-Use Models," 2024. https://groq.com/blog/introducing-llama-3-groq-tool-use-models

[17] N. Muennighoff *et al.*, "OLMoE: Open Mixture-of-Experts Language Models," *arXiv:2409.02060*, 2024. https://huggingface.co/allenai/OLMoE-1B-7B-0924

Document generated 18 May 2026 from the operational cluster. Hellomatik · Raspberry Pi Cluster Lab. Final throughput 14.449 tok/s sustained, +10.81% above public ceiling.